

DOCKER EN LA CIENCIA DE DATOS

Qué es y cómo se utiliza en proyectos de análisis de datos

Asignatura: SCY1101 - Programación para la Ciencia de Datos

Docente: Mariela Moraga

Estudiante: Milenka Guerra

Evaluación Parcial N°3 - Documento técnico

Junio 2026

1. Introducción

En el ciclo de vida de un proyecto de ciencia de datos, uno de los mayores desafíos técnicos es garantizar que el código que funciona en el equipo del desarrollador también funcione correctamente en cualquier otro entorno: el servidor de producción, la máquina de un compañero de equipo o una plataforma de nube.

¿Cuál es el problema?

Si entrenas un modelo en tu dispositivo con Python 3.11 y scikit-learn 1.4 y luego, alguien más intenta ejecutar el mismo notebook en su equipo con Python 3.9 y scikit-learn 1.2, el código fallará por una incompatibilidad de dependencias.

Docker resuelve este problema empaquetando el código junto con su entorno de ejecución completo.

Docker es una plataforma de contenedores que permite empaquetar una aplicación junto con todas sus dependencias, configuraciones y sistema operativo base en una unidad portable y reproducible llamada contenedor. De este modo, el entorno de ejecución es siempre idéntico, sin importar dónde se ejecute.

2. ¿Qué es Docker?

2.1 Definición

Docker es una plataforma de código abierto lanzada en 2013 que automatiza el despliegue de aplicaciones dentro de contenedores de software. Un contenedor es una unidad ligera y ejecutable que incluye:

- El código fuente de la aplicación o notebook.
- El intérprete o runtime necesario (Python, R, etc.).
- Las bibliotecas y dependencias con sus versiones exactas.
- Las variables de entorno y configuraciones del sistema.
- Archivos de datos o scripts de inicialización.

2.2 Docker vs Máquinas Virtuales

Antes de Docker, la solución al problema de entornos era usar máquinas virtuales (VM). Sin embargo, las VM son pesadas, ya que replican un sistema operativo completo para cada instancia. Docker introduce un enfoque más eficiente mediante la virtualización a nivel del sistema operativo.

Máquina Virtual (VM)	Contenedor Docker
Incluye un SO completo (GB de espacio)	Comparte el kernel del SO anfitrión (MB de espacio)
Arranque lento (minutos)	Arranque casi instantáneo (segundos)
Alta demanda de RAM y CPU	Bajo consumo de recursos
Portabilidad limitada	Portable entre cualquier sistema con Docker instalado
Herramienta: VirtualBox, VMware	Herramienta: Docker Engine

2.3 Componentes principales de Docker

Imagen (Image)

Una imagen Docker es una plantilla de solo lectura que define el entorno, como el "molde" a partir del cual se crean los contenedores. Las imágenes se construyen a partir de un archivo de instrucciones llamado **Dockerfile**.

Contenedor (Container)

Un contenedor es una instancia en ejecución de una imagen. Se puede iniciar, detener, eliminar y replicar. Múltiples contenedores pueden ejecutarse desde la misma imagen de forma simultánea e independiente (lo que ayuda en la escalabilidad).

Dockerfile

Archivo de texto con instrucciones para construir una imagen personalizada. Define la imagen base, instala dependencias, copia archivos y establece el comando de inicio.

Docker Hub

Registro público (y privado) de imágenes Docker en la nube. Funciona como un "GitHub de imágenes": del cual se pueden descargar imágenes oficiales de Python, PostgreSQL, Jupyter, entre muchas otras.

Docker Compose

Herramienta para definir y ejecutar aplicaciones multi-contenedor. Usa un archivo YAML (docker-compose.yml) para describir los servicios, redes y volúmenes que componen la solución.

3. Conceptos Clave

3.1 El Dockerfile en detalle

El Dockerfile es el corazón de Docker, donde cada línea representa una capa de la imagen final. A continuación se muestra un ejemplo comentado para un proyecto de ciencia de datos:

```
# 1. Imagen base oficial de Python 3.11 (versión ligera)
FROM python:3.11-slim

# 2. Definir el directorio de trabajo dentro del contenedor
WORKDIR /app

# 3. Copiar el archivo de dependencias primero (optimiza la caché)
COPY requirements.txt .

# 4. Instalar las dependencias de Python
RUN pip install --no-cache-dir -r requirements.txt

# 5. Copiar el resto del proyecto al contenedor
COPY . .

# 6. Exponer el puerto donde correrá Dash/Jupyter
EXPOSE 8050

# 7. Comando que se ejecuta al iniciar el contenedor
CMD ["python", "app.py"]
```

3.2 Instrucciones esenciales del Dockerfile

Instrucción	Descripción	Ejemplo
FROM	Imagen base de partida	FROM python:3.11-slim
WORKDIR	Directorio de trabajo en el contenedor	WORKDIR /app
COPY	Copia archivos del host al contenedor	COPY . .
RUN	Ejecuta un comando durante la construcción	RUN pip install pandas
EXPOSE	Documenta el puerto que usará el contenedor	EXPOSE 8888
ENV	Define variables de entorno	ENV PYTHONUNBUFFERED=1
CMD	Comando por defecto al iniciar el contenedor	CMD ["jupyter", "notebook"]
VOLUME	Define un punto de montaje para datos persistentes	VOLUME /app/data

3.3 Docker Compose para proyectos multi-servicio

En proyectos reales de ciencia de datos, generalmente se necesita más de un servicio: el notebook o script principal, una o varias bases de datos y quizás un dashboard. Docker Compose permite definir todos estos servicios en un solo archivo:

```
# docker-compose.yml para proyecto de análisis de datos
version: "3.9"

services:
  # Servicio principal: aplicación Python/Dash
  app:
    build: .                # Construye desde el Dockerfile local
    ports:
      - "8050:8050"        # Puerto host:contenedor
    volumes:
      - ./data:/app/data   # Monta la carpeta de datos
    environment:
      - PYTHONUNBUFFERED=1
    depends_on:
      - db                 # Espera a que la BD esté lista

  # Servicio de base de datos PostgreSQL
  db:
    image: postgres:15      # Imagen oficial de Docker Hub
    environment:
      POSTGRES_USER: datascience
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: carseats
    volumes:
      - db_data:/var/lib/postgresql/data  # Datos persistentes

volumes:
  db_data:                  # Volumen nombrado para persistencia
```

3.4 Volúmenes: gestión de datos persistentes

Los contenedores son efímeros: cuando se eliminan, pierden sus datos internos. Los volúmenes de Docker solucionan esto montando directorios del host dentro del contenedor, garantizando que los datos (datasets, modelos entrenados, resultados) persistan entre ejecuciones.

Tipos de volúmenes en Docker

Volumen nombrado (named volume): gestionado por Docker, ideal para bases de datos.
Ejemplo: `db_data:/var/lib/postgresql/data`

Bind mount: mapea directamente una carpeta del host al contenedor.
Ejemplo: `./data:/app/data` → permite editar archivos desde el host en tiempo real.

Volumen anónimo: temporal, útil para caché durante la construcción.

4. Docker en Ciencia de Datos

4.1 ¿Por qué usar Docker en proyectos de datos?

La ciencia de datos enfrenta retos únicos de reproducibilidad, debido a que los modelos de machine learning dependen de versiones exactas de frameworks (scikit-learn, TensorFlow, PyTorch) que pueden producir resultados distintos entre versiones. Docker garantiza:

Beneficio	Descripción
Reproducibilidad total	El entorno es exactamente el mismo en cualquier máquina o servidor.
Aislamiento de proyectos	Cada proyecto tiene su propio entorno sin conflictos de versiones.
Colaboración simplificada	El equipo comparte el Dockerfile; nadie necesita instalar nada manualmente.
Despliegue consistente	Lo que funciona en desarrollo funciona idéntico en producción.
Escalabilidad	Con Kubernetes u orquestadores, se pueden replicar contenedores fácilmente.
Integración con CI/CD	Se integra con GitHub Actions para pruebas y despliegues automáticos.

4.2 Casos de uso concretos en Data Science (evaluación 3)

a) Encapsular un pipeline ETL

El pipeline ETL (Extract, Transform, Load) que procesa el dataset Carseats podría ejecutarse dentro de un contenedor Docker como caso general de la industria. En este proyecto, sin embargo, el ETL vive directamente en el notebook (Sección 1 de CarSeats_Ev3.ipynb).

b) Servir un Dashboard con Plotly Dash

El dashboard interactivo de análisis de ventas (`src/dashboard.py`) es el caso más natural para utilizar un contenedor en este proyecto, ya que es el único componente que necesita quedar «corriendo» como servicio. No se implementó en esta entrega, pero el patrón conceptual sería el siguiente:

```
# Patrón conceptual para este proyecto:
# Dockerfile simple, sin docker-compose ni base de datos, ya que el
# dashboard (src/dashboard.py) no depende de ningún servicio externo.

$ docker build -t carseats-dashboard .
$ docker run -p 8050:8050 carseats-dashboard

# El dashboard quedaría disponible en:
# http://localhost:8050
```

c) Entornos Jupyter reproducibles

Existe una imagen oficial de Jupyter en Docker Hub (jupyter/scipy-notebook) que incluye Python, pandas, matplotlib, scikit-learn y más, lista para usarse sin instalación.

```
# Levantar Jupyter Notebook con imagen oficial
$ docker run -p 8888:8888 \
  -v $(pwd)/notebooks:/home/jovyan/work \
  jupyter/scipy-notebook

# Acceder en: http://localhost:8888
```

d) Modelos de Machine Learning en producción

Una vez entrenado el modelo (en este proyecto, Regresión Logística o Random Forest, ambos optimizados para predecir HighSales), puede empaquetarse en un contenedor Docker que expone una API REST con Flask o FastAPI para hacer predicciones en tiempo real.

5. Comandos Docker Esenciales

5.1 Gestión de imágenes

```
# Construir una imagen desde el Dockerfile del directorio actual
$ docker build -t nombre-imagen:version .

# Listar imágenes disponibles localmente
$ docker images

# Descargar una imagen desde Docker Hub
$ docker pull python:3.11-slim

# Eliminar una imagen
$ docker rmi nombre-imagen
```

5.2 Gestión de contenedores

```
# Crear y ejecutar un contenedor en segundo plano (-d)
$ docker run -d -p 8050:8050 --name mi-app nombre-imagen

# Ver contenedores en ejecución
$ docker ps

# Ver TODOS los contenedores (incluidos los detenidos)
$ docker ps -a
```

```
# Detener un contenedor
$ docker stop mi-app

# Iniciar un contenedor detenido
$ docker start mi-app

# Eliminar un contenedor
$ docker rm mi-app

# Ver logs de un contenedor
$ docker logs mi-app

# Acceder a la terminal interactiva de un contenedor
$ docker exec -it mi-app /bin/bash
```

5.3 Docker Compose

```
# Construir y levantar todos los servicios definidos en docker-compose.yml
$ docker-compose up --build

# Levantar en segundo plano (detached)
$ docker-compose up -d

# Ver logs de todos los servicios
$ docker-compose logs -f

# Detener y eliminar contenedores, redes y volúmenes
$ docker-compose down -v

# Ejecutar un comando en un servicio específico
$ docker-compose exec app python etl.py
```

6. Otros archivos relevantes

6.1 El archivo .dockerignore

Similar al .gitignore, el .dockerignore evita copiar archivos innecesarios a la imagen, reduciendo su tamaño y mejorando la seguridad:

```
# .dockerignore (ejemplo ilustrativo)
__pycache__/
*.pyc
.ipynb_checkpoints/
.git/
.env
*.log
.venv/
.DS_Store
```

6.2 El archivo requirements.txt

Es fundamental fijar las versiones exactas de las dependencias para garantizar la reproducibilidad:

```
# requirements.txt con versiones fijadas (buena práctica para Docker).
# Ejemplo ilustrativo:
pandas==2.2.0
numpy==1.26.4
scikit-learn==1.4.0
plotly==5.19.0
dash==2.16.0
requests==2.31.0          # Para consumir la API de tipo de cambio
pyngrok==7.1.6            # Túnel opcional para exponer el dashboard
jupyter==1.0.0
```

7. Flujo de Trabajo con Docker en Data Science

El ciclo de trabajo típico al usar Docker en un proyecto de ciencia de datos es el siguiente:

Paso	Acción	Comando / Archivo
1	Crear el Dockerfile con la imagen base y dependencias	Dockerfile
2	Definir el requirements.txt con versiones exactas	requirements.txt
3	Construir la imagen Docker localmente	docker build -t app .
4	Probar el contenedor en entorno local	docker run -p 8050:8050 app
5	Definir servicios adicionales (BD, API) en Compose	docker-compose.yml
6	Versionar Dockerfile y Compose con Git	git add Dockerfile docker-compose.yml
7	Compartir la imagen en Docker Hub o GitHub Registry	docker push usuario/app:v1
8	Cualquier colaborador reproduce el entorno completo	docker-compose up --build

Buena práctica: reproducibilidad total

El objetivo final es que cualquier persona pueda ejecutar dos comandos para reproducir completamente el proyecto:

```
$ git clone https://github.com/usuario/proyecto-carseats.git
$ docker-compose up --build
```

Y el entorno completo (ETL + Dashboard + Base de datos) se levanta automáticamente.

8. Docker y Git: Integración Profesional

Docker y Git se complementan perfectamente en un flujo de trabajo profesional. Git versiona el código y Docker versiona el entorno, por lo que su combinación garantiza que cualquier commit del repositorio sea completamente reproducible.

Git versiona...	Docker garantiza...
El código fuente (notebooks, scripts)	El entorno de ejecución (Python, librerías)
El Dockerfile y docker-compose.yml	Que el build sea idéntico en cualquier máquina
El historial de cambios del proyecto	La consistencia entre desarrollo y producción
La colaboración entre miembros del equipo	Que todos usen exactamente el mismo entorno

Una práctica recomendada es etiquetar las imágenes Docker con el mismo identificador que los tags de Git, de modo que sea posible rastrear exactamente qué versión del código corresponde a cada imagen:

```
# Etiquetar la imagen con la versión del proyecto (igual que el tag de Git)
$ docker build -t carseats-app:v1.0.0 .

# Al hacer un release en Git, el tag de la imagen coincide:
$ git tag v1.0.0
$ git push origin v1.0.0
```

9. Ventajas y Limitaciones de Docker

9.1 Ventajas

- Reproducibilidad garantizada: el entorno es idéntico en cualquier máquina.
- Portabilidad: la imagen corre en Linux, macOS y Windows (con Docker Desktop).
- Aislamiento: distintos proyectos con versiones de Python incompatibles pueden coexistir.
- Integración con la nube: AWS, Azure y GCP soportan contenedores nativamente.
- Comunidad y ecosistema: miles de imágenes oficiales en Docker Hub.
- Facilita CI/CD: integración nativa con GitHub Actions, Jenkins, GitLab CI.

9.2 Limitaciones y consideraciones

- Curva de aprendizaje inicial: comprender imágenes, capas y redes requiere práctica.
- Uso de disco: las imágenes pueden ocupar varios GB si no se optimizan correctamente.
- Acceso a GPU: requiere configuración adicional (NVIDIA Container Toolkit) para deep learning.
- No es una VM: no reemplaza sistemas operativos completos para propósitos de virtualización general.
- Seguridad: contenedores mal configurados pueden exponer servicios; requiere configuración cuidadosa.

10. Conclusión

Docker representa una herramienta fundamental en el ecosistema moderno de la ciencia de datos. Más allá de ser una tecnología de despliegue, es una práctica de ingeniería que eleva la calidad y profesionalismo de cualquier proyecto de análisis de datos.

Al encapsular el entorno completo de ejecución en un contenedor, Docker elimina la ambigüedad del "funciona en mi máquina" y garantiza que el pipeline ETL, el dashboard interactivo y los modelos de machine learning del proyecto Carseats sean perfectamente reproducibles por cualquier persona.

La combinación de Docker con Git forma la base del flujo de trabajo profesional en ciencia de datos: el código versionado en Git y el entorno empaquetado en Docker permiten que cualquier estado del proyecto sea completamente reproducible en cualquier momento y en cualquier máquina.

Reflexión final

Un proyecto de ciencia de datos que no puede ser reproducido por otros investigadores o ingenieros pierde gran parte de su valor. Docker, junto con Git, es hoy la práctica estándar de la industria para garantizar esa reproducibilidad y facilitar la colaboración profesional en equipos de datos en un mundo cada vez más conectado.

Referencias

- Docker Inc. (2024). Docker Documentation. <https://docs.docker.com>
- Docker Inc. (2024). Docker Hub – Official Python Image. https://hub.docker.com/_/python
- VanderPlas, J. (2022). Python Data Science Handbook (2nd ed.). O'Reilly Media.